

## TP 4 : Transformation de variables, imputation des données manquantes, data leakage

### 0. Ce qui a été fait au TP 2

#### 0.1. Données manquantes

Rappeler les variables du jeu de données `titanic` qui ont des valeurs manquantes, chez combien de passagers, ainsi que la manière dont on pourrait les gérer

*Réponse :*

L'un des buts de ce TP va être d'implémenter des méthodes d'imputation statique et d'imputation prédictive

#### 0.2. Transformation et création de variables

Lors du TP 2 nous avons créé la variable quantitative `Fam` par agrégation en sommant `SibSp` et `Parch`. Nous l'avons ensuite discrétisé pour obtenir `withFam`, une variable binaire indiquant si le passager voyageait avec des membres de sa famille ou non. Nous avons aussi créé une variable catégorielle à partir de la variable `Age` ; les 4 catégories sont basées sur les groupes d'âges : enfants (moins de 13 ans), adolescents (entre 13 et 18 ans), adultes (entre 18 et 60 ans), et seniors (plus de 60 ans)

Dans notre jeu de données nous avons donc une combinaison de variables prédictives quantitatives et qualitatives. Certaines méthodes d'apprentissage supervisé ne peuvent utiliser que des variables prédictives quantitatives, et d'autres que des variables prédictives qualitatives

L'autre but de ce TP va être de créer des fonctions pour implémenter d'autres transformations que nous pourrions utiliser si besoin :

- Le codage binaire (dummy coding)
- Le label encoding
- La normalisation Min-Max
- La standardisation

#### 0.3. Charger le jeu de données

Charger le jeu de données

*Réponse :*

```
qualitative_vars = read.csv("../TP2/titanic_pre_processed_qualitative_vars.csv",
                             stringsAsFactors = TRUE)
quantitative_vars = read.csv("../TP2/titanic_pre_processed_quantitative_vars.csv")
target = read.csv("../TP2/titanic_pre_processed_target.csv")
```

Vérifier que toutes les variables sont bien du bon type

```
qualitative_vars$Pclass = as.factor(qualitative_vars$Pclass)
qualitative_vars$withFam = as.factor(qualitative_vars$withFam)
target = as.factor(target$x)
```

## 1. Données manquantes

### 1.1. Imputation statique

**1.1.1. Pour une variable quantitative** Écrire deux fonctions permettant d'implémenter les valeurs manquantes d'une variable quantitative par sa moyenne ou sa médiane (sans stratification). Essayer les fonctions créées sur la variable `Age`

*Réponse :*

```
mean_imputation = function(v){
  meanv = mean(v, na.rm = TRUE)
  v[is.na(v)] = meanv
  return(v_imput = v)
}

mean_imputation2 = function(v){
  v_imput = v
  v_imput[which(is.na(v_imput))] = mean(v_imput, na.rm = TRUE)
  return(v_imput = v_imput)
}

head(quantitative_vars$Age)
head(mean_imputation(quantitative_vars$Age))

median_imputation = function(v){
  medianv = median(v, na.rm = TRUE)
  v[is.na(v)] = medianv
  return(v_imput = v)
}

median_imputation2 = function(v){
  v_imput = v
  if(sum(is.na(v_imput)) > 0){
    v_imput[which(is.na(v_imput))] = median(v_imput, na.rm = TRUE)
  }
  return(v_imput = v_imput)
}

head(quantitative_vars$Age)
head(median_imputation(quantitative_vars$Age))

#mean_imputation(quantitative_vars$Age)
#median_imputation(quantitative_vars$Age)
```

Rappeler un avantage de la median imputation par rapport à la mean imputation.

*Réponse :* Contrairement à la moyenne, la médiane est moins sensible aux valeurs extrêmes (outliers)

On pourrait aussi utiliser la commande `preProcess` du package `caret` pour faire l'imputation par la médiane

```
library(caret)
```

```
Proc = preProcess(quantitative_vars, method = "medianImpute")
head(predict(Proc, quantitative_vars))
```

L'imputation par la médiane n'est appliquée qu'aux variables avec des valeurs manquantes ; les autres variables sont intouchées

Créer un nouveau dataframe nommé `quantitative_vars_complete` contenant toutes les variables explicatives quantitatives sans valeurs manquantes, notamment la variable nommée `Age_imput_median` dont on a imputé les valeurs manquantes par la médiane de `Age`

```
Proc = preProcess(quantitative_vars, method = "medianImpute")
quantitative_vars_complete = predict(Proc, quantitative_vars)
names(quantitative_vars_complete)[1] = "Age_imput_median" # on utilise un nom qui nous rappelle comment
```

On peut vérifier qu'il n'y a plus de données manquantes après imputation

```
sum(is.na(quantitative_vars$Age)) # on ne connaissait pas l'age de 169 individus
```

```
## [1] 169
```

```
sum(is.na(quantitative_vars_complete$Age_imput_median)) # plus aucune valeur manquante après
```

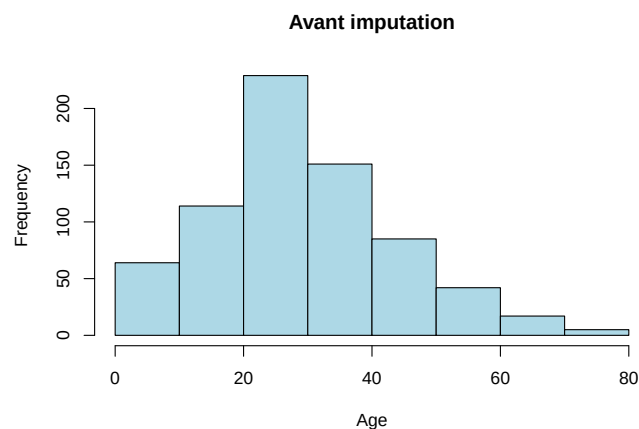
```
## [1] 0
```

```
# imputation par la médiane
```

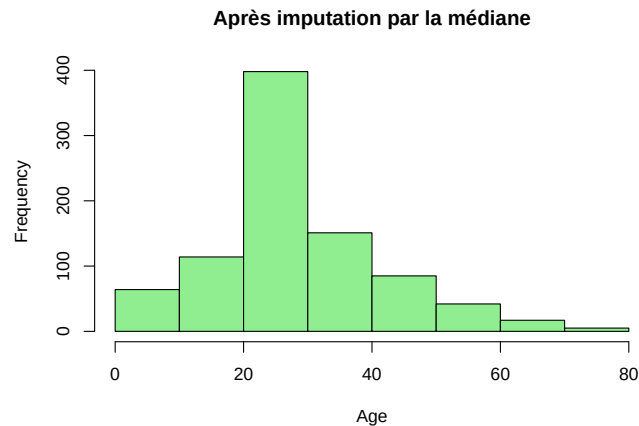
(à faire à la maison) Tracer les histogrammes de la variable avant et après imputation. Que remarque-t-on ?

Réponse :

```
hist(quantitative_vars$Age, main = "Avant imputation", xlab = "Age", col = "lightblue")
```



```
hist(quantitative_vars_complete$Age_imput_median, main = "Après imputation par la médiane", xlab = "Age")
```



### 1.1.2. Pour une variable qualitative Que fait la fonction implémentée ci-dessous ?

```
cat.imputation = function(v){
  if(is.factor(v) == FALSE){
    print("The variable should be a factor")}else{
    max_levels = which.max(table(v)) #stocker l'indice ou on a le plus grand nombre d'occurences (eff
    v_imput = v
    if(sum(is.na(v_imput)) > 0){
      v_imput[which(is.na(v_imput))] = levels(v_imput)[max_levels]
    }
    return(v_imput = v_imput)
  }}
head(cat.imputation(qualitative_vars$Sex))
head(cat.imputation(qualitative_vars$Age_cat))
```

Réponse : cette partie de code permet d'imputer les valeurs manquantes de la variable qualitative par la modalité la plus fréquente, le mode.

On peut renommer la fonction `cat.imputation`

Créer un nouveau dataframe nommé `qualitative_vars_complete` contenant les variables qualitatives sans données manquantes, notamment les variables nommée `Sex_imput_freq` et `Age_cat_imput_freq` dont on a imputé les valeurs manquantes par la valeur la plus fréquente

```
qualitative_vars_complete = qualitative_vars
qualitative_vars_complete$Sex = cat.imputation(qualitative_vars$Sex)
qualitative_vars_complete$Age_cat = cat.imputation(qualitative_vars$Age_cat)
names(qualitative_vars_complete)[3] = "Sex_imput_freq" # on utilise un nom qui nous rappelle comment on
names(qualitative_vars_complete)[1] = "Age_cat_imput_freq"
```

Remarque : Alternativement, on aurait pu catégoriser la variable `Age_imput_mean`

## 1.2. Imputation prédictive par la méthode de régression linéaire (à faire à la maison ou si on a le temps)

Rappeler en une ou deux phrases comment fonctionne l'imputation prédictive de valeurs manquantes par la méthode de régression linéaire. Peut-on imputer les valeurs manquantes de `Age` et `Sex` avec cette méthode ?

*Réponse :* L'imputation prédictive consiste à considérer la variable présentant des données manquantes comme une variable cible () et les autres variables comme des prédicteurs (). On entraîne un modèle sur les individus complets pour prédire les valeurs manquantes des autres. On peut imputer **Age** avec cette méthode (régression linéaire car variable continue), mais pour **Sex**, la régression linéaire n'est pas adaptée car il s'agit d'une variable qualitative binaire.

Quelle commande R que vous connaissez bien utiliserait-on ?

*Réponse :* On utiliserait la commande `lm()` pour ajuster le modèle de régression linéaire.

On voudrait essayer l'implémentation par la méthode de régression linéaire sur la variable **Age**. Que font les deux lignes de code ci-dessous ?

```
isMissing = which(is.na(quantitative_vars$Age))
isNotMissing = which(!is.na(quantitative_vars$Age))
```

*Réponse :* Ces lignes identifient les indices des lignes du dataset : `isMissing` stocke les positions des passagers dont l'âge est inconnu (NA), et `isNotMissing` stocke les positions de ceux dont l'âge est renseigné.

Que font les lignes de code ci-dessous ? *Remarque :* J'ai choisi de n'utiliser que les variables **Fam**, **Fare** pour faire l'imputation

```
age_isNotMissing = quantitative_vars$Age[isNotMissing]
X_isNotMissing = cbind(quantitative_vars[isNotMissing,c(4,5)])
X_isMissing = cbind(quantitative_vars[isMissing,c(4,5)])

fit = lm(age_isNotMissing ~ ., data = X_isNotMissing)

quantitative_vars_complete$Age_imput_reg = quantitative_vars$Age
quantitative_vars_complete$Age_imput_reg[is.na(quantitative_vars_complete$Age_imput_reg)] =
predict(fit, X_isMissing)
```

*Réponse :* Le code extrait d'abord les données cibles et les prédicteurs pour les individus présents et manquants. Il ajuste ensuite un modèle de régression linéaire (`lm`) sur les données connues. Enfin, il crée une nouvelle colonne `Age_imput_reg` et remplace les valeurs manquantes par les prédictions générées par le modèle (`predict`) à partir des variables **Fam** et **Fare**.

### Visualisation pédagogique de l'imputation :

Pour mieux comprendre, nous allons afficher la relation entre le prix du billet (**Fare**) et l'âge. Les points rouges représenteront les valeurs que le modèle a "inventées" (imputées).

```
library(ggplot2)

# Création d'un dataset temporaire pour le plot
plot_data <- data.frame(
  Fare = quantitative_vars$Fare,
  Age = quantitative_vars_complete$Age_imput_reg,
  Status = ifelse(is.na(quantitative_vars$Age), "Imputée", "Réelle")
)

ggplot(plot_data, aes(x = Fare, y = Age, color = Status)) +
  geom_point(alpha = 0.6) +
  scale_color_manual(values = c("Imputée" = "red", "Réelle" = "steelblue")) +
  theme_minimal() +
  labs(title = "Distribution de l'Age après Imputation par Régression",
```

```
subtitle = "Les points rouges suivent la ligne de tendance du modèle",
x = "Tarif (Fare)", y = "Âge")
```

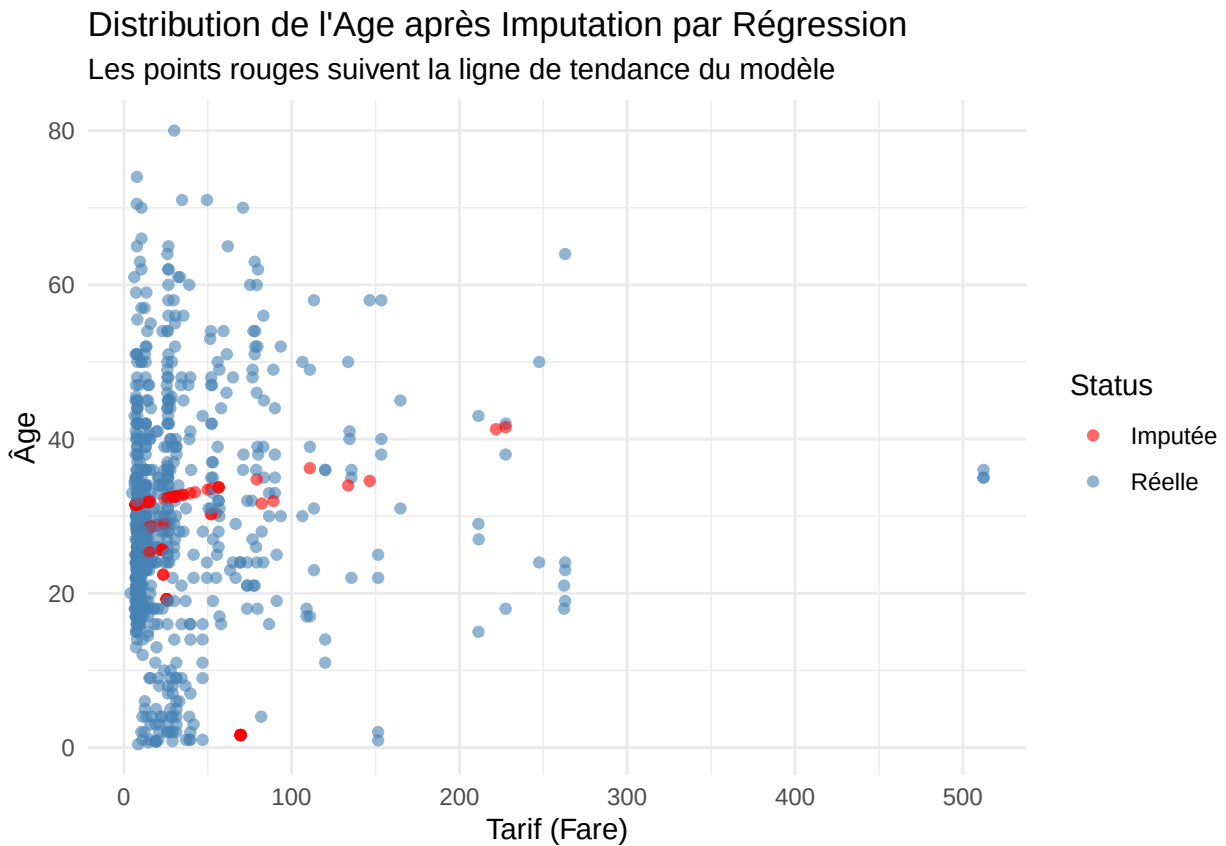


Figure 1: Visualisation de l'imputation par régression : Valeurs réelles vs Imputées

Quelle méthode de régression assez similaire à la régression linéaire pourrait-on utiliser pour faire de l'imputation prédictive sur la variables **Sex** ?

*Réponse :* On utiliserait la **régression logistique** (via la fonction `glm()` avec l'argument `family = "binomial"`), qui est l'équivalent de la régression linéaire pour les variables de nature binaire.

### 1.3. Création d'une catégorie pour les NA

Lire la documentation de la commande `addNA`. Dans le dataframe `qualitative_vars_complete`, créer une nouvelle variable nommée `Age_cat_missing` contenant une catégorie spécialement pour les individus dont la valeur de l'âge était manquante.

```
qualitative_vars_complete$Age_cat_missing = addNA(qualitative_vars$Age_cat)
head(qualitative_vars_complete$Age_cat_missing)
head(qualitative_vars_complete$Age_cat_imput_freq)
```

## 2. Transformation de variables

### 2.1. Codage binaire (dummy coding)

Rappeler en 1 ou 2 phrases ce qu'est le codage binaire et sur quel type de variables on peut l'utiliser

*Réponse :* Le codage binaire consiste à transformer une variable catégorielle en plusieurs variables indicatrices (0/1), une par catégorie. On l'utilise sur des variables qualitatives nominales ou ordinales.

Pourquoi peut-on être amené à utiliser le codage binaire ?

*Réponse :* Certaines méthodes d'apprentissage ne gèrent que des variables numériques. Le codage binaire permet d'intégrer des variables qualitatives sans imposer d'ordre arbitraire.

Qu'est-ce qu'une "dummy variable" ?

*Réponse :* C'est une variable binaire valant 1 si l'observation appartient à une catégorie donnée et 0 sinon.

Nous allons utiliser le package `caret`. Lire la documentation de la commande `dummyVars`

Appliquer le codage binaire aux variables `Age_cat_missing` et `Age_cat_imput_freq` du dataframe `qualitative_vars_complete`, et vérifier que les variables créées possèdent bien les propriétés désirées. Combien de variables binaires ont été créées ? Est-ce que c'était attendu ?

*Réponse :*

```
library(caret)
dummy = dummyVars(~ Age_cat_missing, data = qualitative_vars_complete)
head(qualitative_vars_complete$Age_cat_missing)
head(predict(dummy, newdata = qualitative_vars_complete))

dummy2 = dummyVars(~ Age_cat_imput_freq, data = qualitative_vars_complete)
head(qualitative_vars_complete$Age_cat_imput_freq)
head(predict(dummy2, newdata = qualitative_vars_complete))
```

*Réponse :* On obtient une variable binaire (dummy) par catégorie de la variable d'origine. Chacune de ces variables est bien de type numérique. Dans le cas où une catégorie avait été créée pour les individus avec des valeurs manquantes, une cinquième variable binaire a été créée.

Appliquer le codage binaire à la variable `Age_cat` du dataframe `qualitative_vars`. Que remarque-t-on ?

*Réponse :* Les lignes avec des valeurs manquantes produisent des NA dans les variables binaires créées.

```
dummy = dummyVars(~ Age_cat, data = qualitative_vars)
head(predict(dummy, newdata = qualitative_vars))
```

*Réponse :* il y a des valeurs manquantes dans les dummy variables créées

Que préconiser dans ce cas ?

*Réponse :* Il faut gérer les données manquantes (imputation, création d'une catégorie missing, etc.) avant d'appliquer le codage binaire

Serait-il nécessaire d'appliquer le codage binaire à la variable `withFam` ?

*Réponse :* La variable `withFam` contient seulement deux catégories nommées 0 et 1, donc on pourrait l'utiliser directement comme une variable numérique (après conversion en `num`) par exemple avec la méthode des K plus proches voisins.

```
# as.numeric(as.character(qualitative_vars_complete$withFam))
# on a besoin du as.character pour éviter que R utilise les indices des niveaux du facteur
```

Appliquer le codage binaire à `Sex_imput_freq` du dataframe `qualitative_vars_complete`. Il y a-t-il une autre façon de faire que d'employer le codage binaire pour transformer `Sex_imput_freq` et pouvoir l'utiliser comme une variable numérique ?

*Réponse :* Oui. Comme `Sex_imput_freq` est binaire, on peut créer une seule variable indicatrice (par exemple 1 si femme, 0 sinon) sans passer par le codage binaire complet.

```
Woman = 1 *(qualitative_vars_complete$Sex_imput_freq == "female")
```

*Réponse :*

## 2.2. Label encoding

Rappeler en 1 ou 2 phrases ce qu'est le label encoding et sur quel type de variables on peut l'utiliser

*Réponse :* Le label encoding est une technique de transformation des variables qualitatives en variables quantitatives en attribuant un entier unique à chaque catégorie. On peut l'utiliser sur des variables qualitatives ordinales ou nominales.

Pourquoi peut-on être amené à utiliser le label encoding ?

*Réponse :* Certaines méthodes comme par exemple la méthode des K plus proches voisins n'acceptent que des variables numériques. En employant le label encoding on peut transformer une variable catégorielle en associant une valeur numérique à chacune de ses catégories. Ainsi la variable créée peut être traitée comme une variable numérique, et on obtient moins de variables qu'avec la méthode de codage binaire. Mais cela ne fait sens que pour les variables catégorielles ordinales puisqu'il faut hiérarchiser les catégories

Nous pouvons utiliser la commande `LabelEncoder` du package `superml` (ou si vous souhaitez écrire votre propre fonction, tout simplement utiliser les commandes `factor`, et `as.numeric`)

```
install.packages("superml")
```

```
#library(superml)
```

Appliquer le label encoding à la variable `Pclass` du dataframe `qualitative_vars_complete`, et vérifier que la variable créée possède bien les propriétés désirées

*Réponse :*

```
#sans la library
as.numeric(qualitative_vars_complete$Age_cat_imput_freq)
as.numeric(qualitative_vars_complete$Pclass)
```

Que se passe-t-il si on applique le label encoding à une variable contenant des données manquantes ?

*Réponse :*

```
#lbl$fit_transform(qualitative_vars$Age_cat)
```



*Réponse* : Les NA restent NA (ils ne sont pas convertis en catégorie).

Que préconiser dans ce cas ?

*Réponse* : Si l'on souhaite imputer les données manquantes, il faut donc le faire avant d'appliquer le label encoding

Est-ce que ça ferait sens d'appliquer le label encoding à la variable `Age_cat_imput_freq` du dataframe `qualitative_vars_complete` ? Et à `Age_cat_missing` ?

*Réponse* : Dans "`Age_cat_imput_freq`", les valeurs manquantes dans la variable "`Age_cat`" ont été imputées; cela pourrait donc faire sens de lui appliquer le label encoding puisque qu'on peut ranger les catégories par exemple par ordre croissant d'âge. Mais dans "`Age_cat_missing`" une catégorie a été créée pour les NA, et on ne saurait pas comment l'ordonner par rapport aux autres

### 2.3. Normalisation Min-Max

Rappeler en 1 ou 2 phrases ce qu'est la normalisation Min-Max et sur quel type de variables on peut l'utiliser

*Réponse* : c'est une technique de mise à l'échelle des variables quantitatives qui transforme les valeurs pour qu'elles se situent dans un intervalle spécifique, généralement  $[0, 1]$ . On peut l'utiliser sur des variables quantitatives continues.

Pourquoi peut-on être amené à utiliser la normalisation Min-Max ?

*Réponse* : Certaines méthodes comme par exemple la méthode des K plus proches voisins sont sensibles à l'échelle des variables prédictives numériques. Dans ce cas il est important de normaliser ou standardiser les variables explicatives avant d'utiliser la méthode

Lire la documentation de la commande `preProcess` du package `caret`. Que font les deux lignes de code ci-dessous ?

```
Proc = preProcess(quantitative_vars, method = "range")
head(predict(Proc, quantitative_vars))
```

*Réponse* : On applique la normalisation Min-Max à chacune des variables ; les nouvelles variables ont toutes les valeurs comprises entre 0 et 1

Que se passe-t-il si on applique la normalisation Min-Max à une variable contenant des données manquantes ? Que préconiser dans ce cas ?

*Réponse* : Les valeurs manquantes restent NA et la transformation peut échouer ou propager des NA. Il faut d'abord imputer (ou traiter) les valeurs manquantes avant la normalisation.

Peut-on appliquer une telle normalisation à une variable label encodée ?

*Réponse* : Oui, mais cela n'est pertinent que si la variable est ordinale (les valeurs ont un ordre réel). Pour une variable nominale, la normalisation peut induire une fausse notion de distance.

### 2.4. Standardisation

Rappeler en 1 ou 2 phrases ce qu'est la standardisation et sur quel type de variables on peut l'utiliser

*Réponse* : La standardisation consiste à centrer et réduire une variable (moyenne 0, écart-type 1). On l'applique aux variables quantitatives continues.

Pourquoi peut-on être amené à utiliser la standardisation ?

*Réponse* : Pour rendre les variables comparables en échelle et éviter qu'une variable à grande variance domine les méthodes sensibles aux distances (KNN, SVM, etc.).

Lire la documentation de la commande `scale` du package `base`. L'essayer sur une des variables du jeu de données, par exemple `Fare`, et vérifier que la variable créée possède les propriétés désirées

*Réponse :*

```
head(quantitative_vars$Fare)
head(scale(quantitative_vars$Fare))
mean(scale(quantitative_vars$Fare))
sd(scale(quantitative_vars$Fare))
```

Que se passe-t-il si on applique la standardisation à une variable contenant des données manquantes ? Que préconiser dans ce cas ?

```
head(scale(quantitative_vars$Age))
```

*Réponse :* Pour les mêmes raisons que dans les sections précédentes, il est recommandé de faire l'imputation des valeurs manquantes de la variable avant d'appliquer la standardisation

Peut-on appliquer la standardisation à une variable label encodée ?

*Réponse :* C'est possible car l'intérêt du label encoding est de pouvoir traiter la nouvelle variable comme une variable numérique et non pas catégorielle. On peut donc appliquer la standardisation, pour que, par exemple dans la méthode des K plus proches voisins, elle soit à la même échelle que les variables quantitatives standardisées.

Que font les deux lignes de code ci-dessous ?

*Indication :* Comparer les valeurs de `Fare` dans `quantitative_vars_stand` et celles obtenues avec la commande `scale`

```
Proc = preProcess(quantitative_vars_complete, method = c("center", "scale"))
quantitative_vars_stand = predict(Proc, quantitative_vars_complete)
head(quantitative_vars_stand)
```

| ##   | Age_imput_median | SibSp      | Parch      | Fam         | Fare       | Age_imput_reg |
|------|------------------|------------|------------|-------------|------------|---------------|
| ## 1 | -0.5596463       | 0.4216511  | -0.4783589 | 0.04924004  | -0.5107583 | -0.5713848    |
| ## 2 | 0.6623190        | 0.4216511  | -0.4783589 | 0.04924004  | 0.7715278  | 0.6236972     |
| ## 3 | -0.2541550       | -0.4792425 | -0.4783589 | -0.56696393 | -0.4972413 | -0.2726143    |
| ## 4 | 0.4332005        | 0.4216511  | -0.4783589 | 0.04924004  | 0.4074017  | 0.3996193     |
| ## 5 | 0.4332005        | -0.4792425 | -0.4783589 | -0.56696393 | -0.4947381 | 0.3996193     |
| ## 6 | -0.1014093       | -0.4792425 | -0.4783589 | -0.56696393 | -0.4865618 | 0.1420600     |

*Réponse :* On estime, à partir de `quantitative_vars_complete`, la moyenne et l'écart-type de chaque variable puis on crée des versions standardisées (moyenne 0, écart-type 1).

## 4. Data leakage

### 4.1. Définition

Partager le jeu de données en train et test par la méthode du holdout. Considérer la réponse `target` et par exemple l'ensemble de variables explicatives dans `quantitative_vars_stand`

```
holdOut = function(X, y, split = 0.8){
  n      = nrow(X)
  inTrain = sample(1:n, size = round(split * n), replace = FALSE)
  X_train = X[inTrain,]
  y_train = y[inTrain]
  X_test  = X[-inTrain,]
  y_test  = y[-inTrain]
  return(list(X_train = X_train, y_train = y_train, X_test = X_test, y_test = y_test, inTrain = inTrain))
}
```

Si maintenant on souhaite utiliser la partie Train pour faire l'apprentissage d'un classifieur supervisé (par exemple K plus proches voisins, naïve Bayes, etc), est-il bien vrai qu'aucune information provenant de la partie Test ne sera utilisée ?

*Indication :* Comment a-t-on obtenu (par exemple) la variables `Age_imput_median` ou `Age_imput_reg` ? Quelles observations ont été utilisées pour faire (i) l'imputation (e.g., estimer les coefficient du modèle de régression linéaire, calculer la médiane), et (ii) pour estimer la moyenne et la variance qu'on utilise ensuite pour la standardisation des variables ?

*Réponse :* Non, car certaines étapes de prétraitement (imputation, standardisation, etc.) ont été apprises sur tout le jeu de données, donc elles utilisent aussi l'information de la partie Test.

Quel est le problème ?

*Indication :* Chercher “train-test contamination” et/ou data leakage dans Google

*Réponse :* C'est une fuite d'information (data leakage), qui biaise l'évaluation en rendant les performances trop optimistes.

## 4.2. Solution

Que proposer pour éviter ce problème ? codage binaire ou label coding

*Réponse :* Séparer d'abord Train/Test, puis apprendre toutes les transformations uniquement sur Train et les appliquer à Test. Pour le codage binaire/label encoding, il faut construire les niveaux sur Train et gérer les niveaux inconnus dans Test (par exemple en les regroupant dans une catégorie “Autre”).

```
data_split = holdOut(X = quantitative_vars, y = target, split = 0.8)

quantitative_vars_train = data_split$X_train
quantitative_vars_test  = data_split$X_test
```

*Remarque :* Attention, si on divise notre jeu de données en Train et Test séparément pour les variables quantitatives et qualitatives, il faut bien que les mêmes individus soient placés dans le Train pour chaque pour les variables quantitatives et qualitatives

Par exemple, considérons l'imputation par la médiane (ou moyenne) des valeurs manquantes d'une variable quantitative, il faudrait :

- Estimer la médiane (ou moyenne) des valeurs non manquantes chez les individus dans la partie Train, puis imputer les valeurs manquantes dans Train.
- Imputer les valeurs manquantes dans Test avec la médiane (ou moyenne) estimée sur Train.

Mettre cela en place en s'appuyant sur le code où on avait utilisé la commande `preProcess` du package `caret` pour faire l'imputation par la médiane

Réponse :

```
Proc = preProcess(quantitative_vars_train, method = c("medianImpute"))
quantitative_vars_complete_train = predict(Proc, quantitative_vars_train)
quantitative_vars_complete_test = predict(Proc, quantitative_vars_test)
```

Ensuite, considérons par exemple la normalisation Min-Max, il faudrait :

- Calculer le min et le max (de chaque variable) dans Train
- Appliquer la même transformation Min-Max, c'est à dire utilisant le minimum et le maximum dans train à la partie test

Mettre cela en place en s'appuyant sur le code où on avait utilisé la commande `preProcess` du package `caret` pour faire l'implémentation Min-Max :

Réponse :

```
Proc = preProcess(quantitative_vars_complete_train, method = "range")
quantitative_vars_normed_train = predict(Proc, quantitative_vars_complete_train)
quantitative_vars_normed_test = predict(Proc, quantitative_vars_complete_test)
```

Pour la standardisation, il faudrait :

- Estimer la moyenne et la variance (de chaque variable) dans Train
- Appliquer la même standardisation, c'est à dire utilisant la moyenne et variance dans Train, à la partie Test

Mettre cela en place en s'appuyant sur le code où on avait utilisé la commande `preProcess` du package `caret` pour faire la standardisation :

Réponse :

```
Proc = preProcess(quantitative_vars_normed_train, method = c("center", "scale"))
quantitative_vars_stand_train = predict(Proc, quantitative_vars_normed_train)
quantitative_vars_stand_test = predict(Proc, quantitative_vars_normed_test)
```

Une “train-test contamination” est-elle aussi possible avec du dummy coding ou du label encoding ?

Réponse :

## 5. Résumé

Résumer ci-dessous les grandes étapes à suivre avant de pouvoir appliquer une méthode de classification supervisée à nos données :

*Séparer le jeu de données en une partie Train et une partie Test (pour faire l'apprentissage, et pour évaluer les performances du classifieur obtenu en termes de prédiction)* Gérer les données manquantes. Si on souhaite imputer il faut le faire de manière séparée pour le Train et le Test : le faire tout d'abord sur le Train, puis appliquer la même imputation sur le Test (en utilisant la médiane sur le Train, les coefficients de régression linéaire estimés sur le Train, etc.) \* Transformer les variables explicatives si besoin. Pour la standardisation

ou la normalisation Min-Max, il faut le faire de manière séparée pour le Train et le Test : le faire tout d'abord sur le Train, puis appliquer la même transformation sur le Test (en utilisant le minimum et le maximum estimés sur le Train, la moyenne et la variance estimées sur le Train, etc.)

Si on choisit de faire de la validation croisée k-fold, faut-il effectuer cela à chacune des itérations ?

*Réponse* : Oui, sinon on s'expose à une contamination